

DEPARTMENT OF MECHANICAL ENGINEERING

MECHATRONICS & IOT 24MEC51

EXERCISE NO: 1

GURUTHARSAN S	- 24MER011
HARISH KUMAR K K	- 24MER015
MADHANRAJ K	- 24MER028
SADAAGOPAAN T R	- 24MER052

MECHATRONICS & IoT

ASSIGNMENT REPORT – 1

TITLE:

Design and simulate a Smart Flood Level Monitoring System using Ultrasonic Sensor. The system continuously monitors water levels and provides alerts during critical flood conditions for disaster management applications.

Done By:

GURTHARSAN S - 24MER011

HARISH KUMAR K K – 24MER015

MADHANRAJ K – 24MER028

SADAAGOPAAN T R – 24MER052

Project Overview:

The Smart Flood Level Monitoring System using an Ultrasonic Sensor is designed to continuously monitor water levels and provide early warning alerts during critical flood conditions. The system is intended for disaster management applications such as rivers, dams, canals, reservoirs, and flood-prone areas.

An HC-SR04 ultrasonic sensor is used to measure the distance between the sensor and the water surface without making physical contact with the water. An Arduino Uno processes the sensor data and calculates the water level.

Based on predefined threshold levels, the system provides three different conditions:

- Normal: Green LED
- Warning: Yellow LED
- Critical: Red LED + Buzzer

The proposed system can be simulated and tested using platforms such as Tinkercad, allowing the circuit and program to be verified before physical prototype implementation.

Problem Statement

Floods are one of the major natural disasters that can cause loss of life, damage to infrastructure, agricultural losses, and displacement of people.

Traditional flood monitoring methods often depend on manual observation or periodic measurements. These methods may not provide sufficient warning when water levels increase rapidly.

The major problems are:

- Lack of continuous water-level monitoring.

- Delayed detection of dangerous water levels.
- Dependence on manual monitoring.
- Lack of immediate warning systems.
- Difficulty in monitoring remote locations.
- Risk to personnel who manually inspect flood-prone areas.

Therefore, a smart and low-cost system is required to continuously monitor water levels and automatically generate alerts when critical flood conditions occur.

Proposed Solution

The proposed system consists of an HC-SR04 ultrasonic sensor, Arduino Uno, LEDs, and a buzzer.

The ultrasonic sensor is mounted above the water surface. It emits ultrasonic waves toward the water surface and receives the reflected signal.

The Arduino measures the time taken for the ultrasonic wave to travel to the water surface and return.

The distance is calculated using:

$$\text{Distance} = (\text{Echo Time} \times \text{Speed of Sound}) / 2$$

The water level is then calculated as:

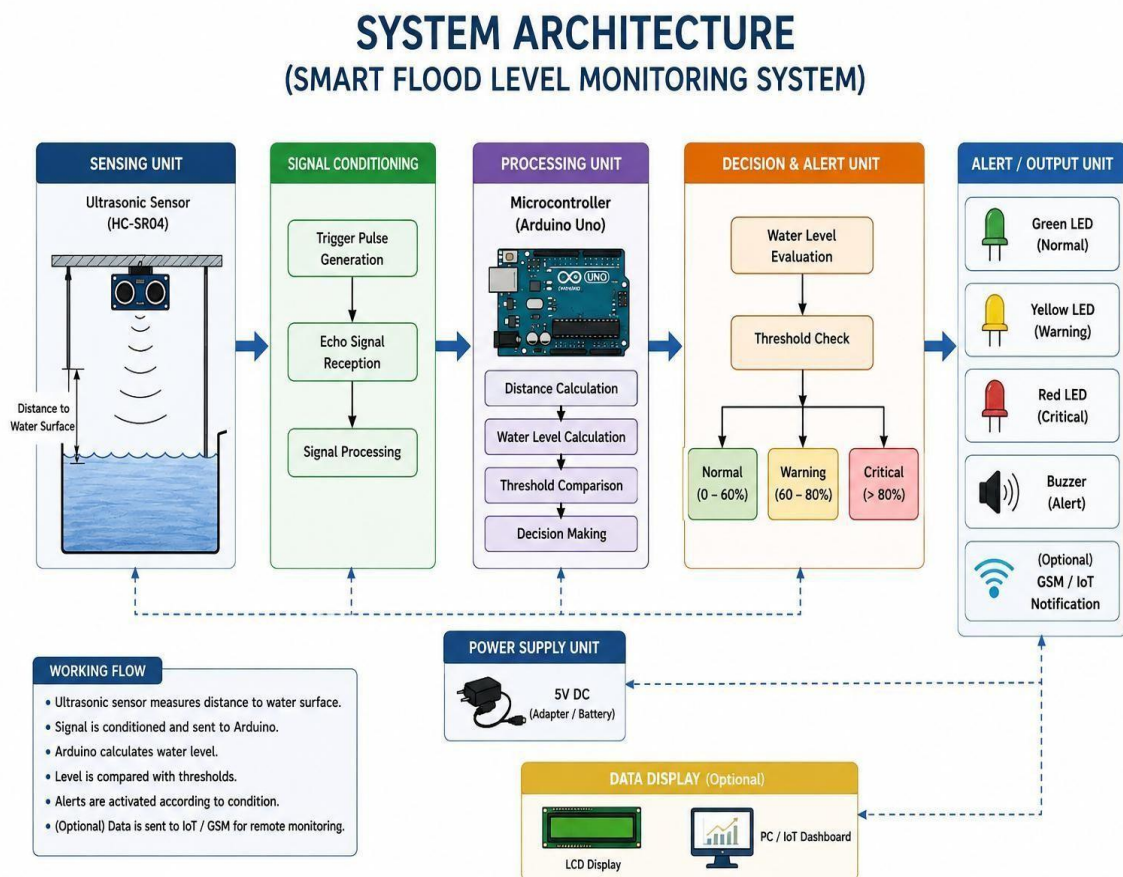
$$\text{Water Level} = \text{Reference Height} - \text{Measured Distance}$$

The calculated water level is compared with predefined threshold values.

Water Level	Condition	Alert
Below 60%	Normal	Green LED
60–80%	Warning	Yellow LED
Above 80%	Critical	Red LED + Buzzer

When the water level exceeds the critical threshold, the system immediately activates the buzzer to warn people about the possible flood condition.

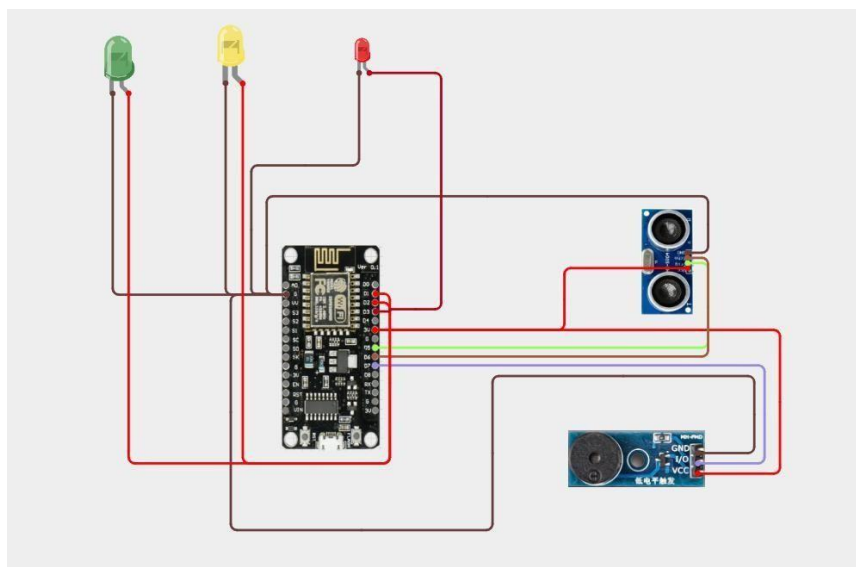
System Architecture



Circuit Table

Component	Pin	Arduino Connection
HC-SR04	VCC	5V
HC-SR04	GND	GND
HC-SR04	TRIG	Digital Pin 9
HC-SR04	ECHO	Digital Pin 10
Green LED	Anode	Digital Pin 2 through 220 Ω
Green LED	Cathode	GND
Yellow LED	Anode	Digital Pin 3 through 220 Ω
Yellow LED	Cathode	GND
Red LED	Anode	Digital Pin 4 through 220 Ω
Red LED	Cathode	GND
Buzzer	Positive Digital	Pin 5
Buzzer	Negative	GND

Circuit Design



Algorithm

1. START
2. Initialize Arduino Uno.
3. Initialize HC-SR04 Ultrasonic Sensor.
4. Initialize Green LED, Yellow LED, Red LED and Buzzer.
5. Set the reference height of the water monitoring area.
6. Send a trigger pulse from the ultrasonic sensor.
7. Receive the reflected echo signal from the water surface.
8. Measure the echo time.
9. Calculate the distance between the sensor and water surface.

$$\text{Distance} = (\text{Echo Time} \times 0.0343) / 2$$

10. Calculate the water level.

$$\text{Water Level} = \text{Reference Height} - \text{Distance}$$

11. Calculate the water level percentage.

$$\text{Water Level \%} = (\text{Water Level} / \text{Reference Height}) \times 100$$

12. Compare the calculated water level percentage with the predefined threshold values.

13. If Water Level < 60%:

Turn ON Green LED.

Turn OFF Yellow LED, Red LED and Buzzer.

Display "NORMAL".

14. Else if Water Level $\geq$ 60% and $\leq$ 80%:

Turn ON Yellow LED.

Turn OFF Green LED, Red LED and Buzzer.

Display "WARNING".

15. Else if Water Level > 80%:

Turn ON Red LED.

Turn ON Buzzer.

Turn OFF Green LED and Yellow LED.

Display "CRITICAL FLOOD ALERT".

16. Display the measured distance, water level and water level percentage.

17. Repeat the measurement continuously.

18. Continue monitoring until the system is switched OFF.

19. STOP

Coding

```
/******
```

Water Level Detection & Flood Alert System

NodeMCU ESP8266

```
*****/
```

```
#include <ESP8266WiFi.h>
```

```
#include "AdafruitIO_WiFi.h"
```

```
// WiFi Credentials
```

```
#define WIFI_SSID "YOUR_WIFI_NAME"
```

```
#define WIFI_PASS "YOUR_WIFI_PASSWORD"
```

```
// Adafruit IO Credentials
```

```
#define IO_USERNAME "YOUR_ADAFRUIT_USERNAME"
```

```
#define IO_KEY "YOUR_ADAFRUIT_KEY"
```

```
AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID,  
WIFI_PASS);
```

```
// Adafruit IO Feeds
```

```
AdafruitIO_Feed *waterLevelFeed = io.feed("water-level");
```

```
AdafruitIO_Feed *distanceFeed = io.feed("distance");
```

```
AdafruitIO_Feed *statusFeed = io.feed("status");
```

```
// GPIO Pins
```

```
#define TRIG_PIN 14 // GPIO14 (D5)
```

```
#define ECHO_PIN 12 // GPIO12 (D6)
```

```
#define GREEN_LED 5 // GPIO5 (D1)
```

```
#define YELLOW_LED 4 // GPIO4 (D2)
```

```
#define RED_LED 13 // GPIO13 (D7)
```

```
#define BUZZER 15 // GPIO15 (D8)
```

```
// Tank Height
```

```
const float tankHeight = 100.0; // cm
```

```
bool alertSent = false;
```

```
unsigned long lastUpload = 0;
```

```
void setup() {  
  
    Serial.begin(115200);  
  
    pinMode(TRIG_PIN, OUTPUT);  
    pinMode(ECHO_PIN, INPUT);  
  
    pinMode(GREEN_LED, OUTPUT);  
    pinMode(YELLOW_LED, OUTPUT);  
    pinMode(RED_LED, OUTPUT);  
    pinMode(BUZZER, OUTPUT);  
  
    digitalWrite(GREEN_LED, LOW);  
    digitalWrite(YELLOW_LED, LOW);  
    digitalWrite(RED_LED, LOW);  
    digitalWrite(BUZZER, LOW);  
  
    Serial.println("Connecting to Adafruit IO...");  
  
    io.connect();  
  
    while (io.status() < AIO_CONNECTED) {  
        Serial.print(".");  
        delay(500);  
    }  
}
```

```
Serial.println();  
Serial.println("Adafruit IO Connected!");  
}  
  
void loop() {  
  
    io.run();  
  
    float distance = readDistance();  
    float level = ((tankHeight - distance) / tankHeight) * 100.0;  
    level = constrain(level, 0, 100);  
  
    String status = "SAFE";  
  
    if (level < 40) {  
  
        digitalWrite(GREEN_LED, HIGH);  
        digitalWrite(YELLOW_LED, LOW);  
        digitalWrite(RED_LED, LOW);  
        digitalWrite(BUZZER, LOW);  
  
        status = "SAFE";  
        alertSent = false;  
    }  
}
```

```
else if (level < 80) {
```

```
    digitalWrite(GREEN_LED, LOW);  
    digitalWrite(YELLOW_LED, HIGH);  
    digitalWrite(RED_LED, LOW);  
    digitalWrite(BUZZER, LOW);
```

```
    status = "WARNING";
```

```
    alertSent = false;
```

```
}
```

```
else {
```

```
    digitalWrite(GREEN_LED, LOW);  
    digitalWrite(YELLOW_LED, LOW);  
    digitalWrite(RED_LED, HIGH);  
    digitalWrite(BUZZER, HIGH);
```

```
    status = "DANGER";
```

```
    if (!alertSent) {
```

```
        Serial.println("SMS Sent: Danger Alert");
```

```
        alertSent = true;
```

```
    }
```

```
}
```

```
// Serial Monitor
```

```
Serial.print("Distance: ");
```

```
Serial.print(distance, 1);
```

```
Serial.print(" cm | Level: ");
```

```
Serial.print(level, 1);
```

```
Serial.print("% | Status: ");
```

```
Serial.println(status);
```

```
// Upload to Adafruit IO every 5 seconds
```

```
if (millis() - lastUpload >= 5000) {
```

```
    waterLevelFeed->save(level);
```

```
    distanceFeed->save(distance);
```

```
    statusFeed->save(status);
```

```
    lastUpload = millis();
```

```
}
```

```
delay(300);
```

```
}
```

```
// Ultrasonic Function
```

```
float readDistance() {
```

```
digitalWrite(TRIG_PIN, LOW);  
delayMicroseconds(2);  
  
digitalWrite(TRIG_PIN, HIGH);  
delayMicroseconds(10);  
  
digitalWrite(TRIG_PIN, LOW);  
  
long duration = pulseIn(ECHO_PIN, HIGH, 30000);  
  
if (duration == 0)  
    return tankHeight;  
  
return duration * 0.0343 / 2.0;  
}
```

System Working

The system continuously measures the distance between the ultrasonic sensor and the water surface.

When the water level is low, the distance measured by the sensor is relatively large. The Arduino calculates a low water-level percentage and activates the green LED.

As water rises, the distance between the sensor and water surface decreases. The calculated water-level percentage increases.

When the water level reaches the warning range, the Arduino activates the yellow LED.

When the water reaches the critical threshold, the Arduino activates both the red LED and buzzer.

This provides an early warning that can be used in disaster-management applications

Bills Of Material

S.No.	Component	Specification	Quantity	Purpose
1	Arduino Uno	ATmega328P	1	Main controller
2	Ultrasonic Sensor	HC-SR04	1	Water level measurement
3	Green LED	5 mm	1	Normal condition indication
4	Yellow LED	5 mm	1	Warning condition indication
5	Red LED	5 mm	1	Critical condition indication
6	Resistor	220 Ω	3	LED current limiting
7	Buzzer	5 V	1	Flood alert
8	Breadboard	Standard	1	Circuit assembly
9	Jumper Wires	Male-Male	1 Set	Circuit connections
10	Power Supply	5 V DC / USB	1	System power supply
11	Water Container	Transparent	1	Prototype water tank

Prototype Implementation

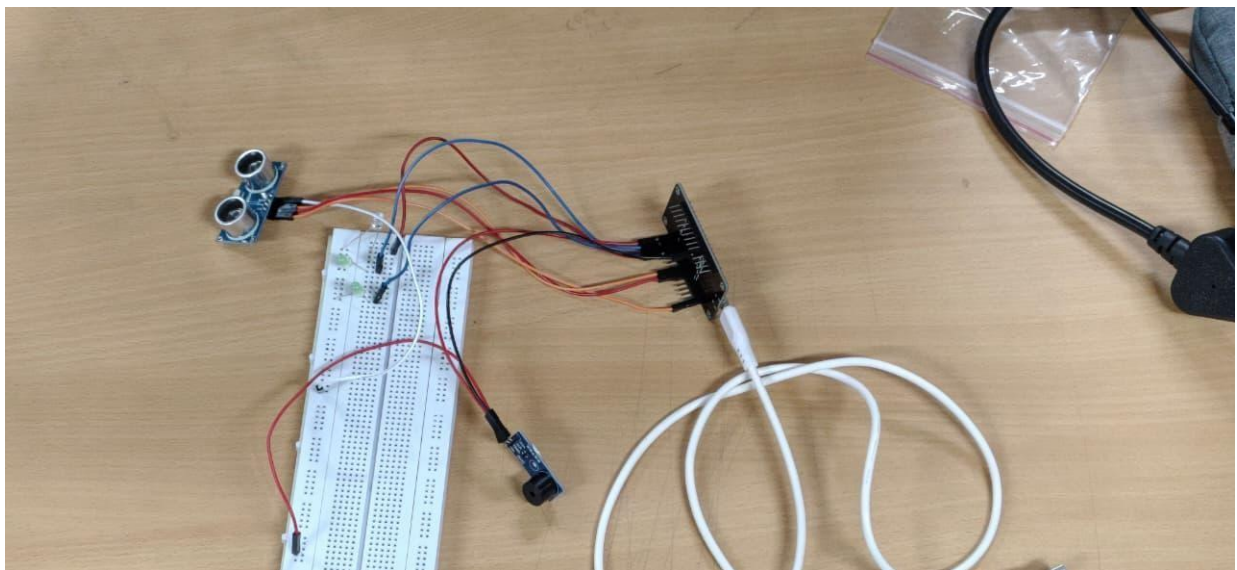
After successful simulation, the system can be implemented as a physical prototype.

The Arduino Uno and electronic components are placed on a breadboard. The HC-SR04 sensor is mounted above a transparent water container.

The sensor is positioned perpendicular to the water surface. LEDs are placed on the front of the prototype so that the user can easily identify the water condition.

The buzzer is connected to the Arduino and programmed to activate when the water level exceeds the critical threshold.

Water is gradually added to the container to test the system under different conditions.



Results & Performance Analysis

The system can be tested by changing the water level and observing the corresponding output.

For a prototype reference height of 30 cm, the expected results are:

Water Level	Percentage	Condition	Output
5 cm	16.7%	Normal	Green LED
10 cm	33.3%	Normal	Green LED
15 cm	50%	Normal	Green LED
18 cm	60%	Warning	Yellow LED
21 cm	70%	Warning	Yellow LED
24 cm	80%	Critical	Red LED + Buzzer
27 cm	90%	Critical	Red LED + Buzzer

Performance Analysis

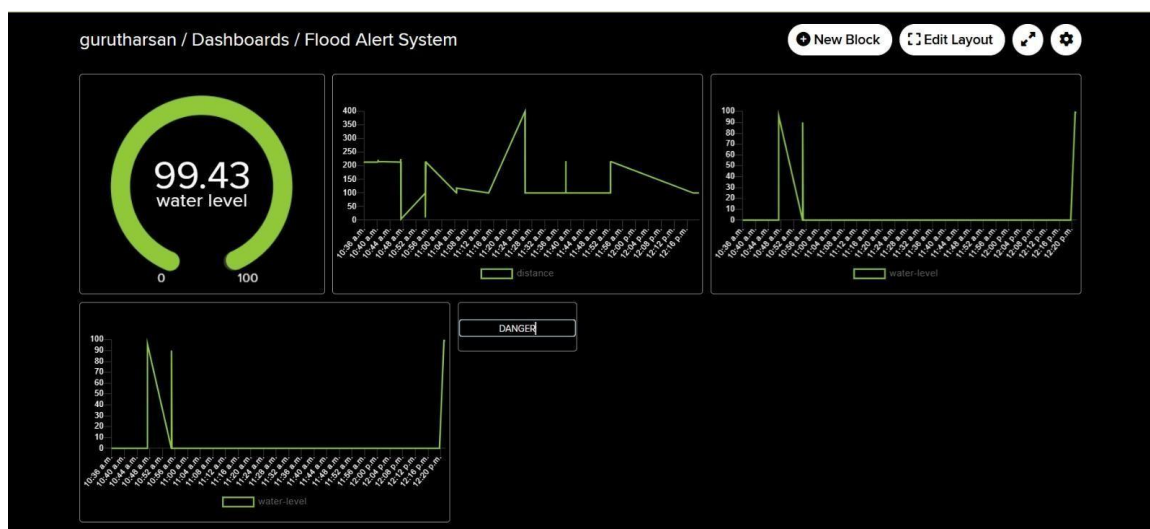
The system provides continuous monitoring and successfully identifies different water-level conditions.

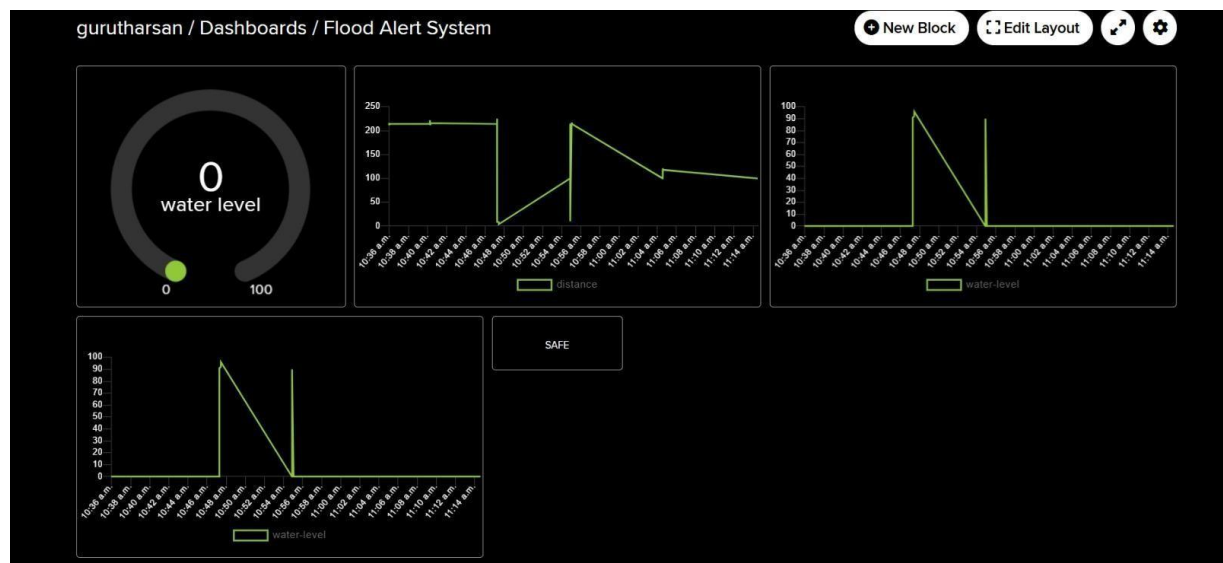
The ultrasonic sensor provides non-contact measurement, reducing the possibility of sensor damage caused by direct contact with water.

The three-level warning system provides a simple and effective method for identifying the severity of the water condition.

The prototype is particularly suitable for demonstrating the basic concept of automated flood-level monitoring.

However, actual flood-monitoring installations require more robust sensors and communication systems because real rivers and dams may have waves, debris, rain, wind, and other environmental factors.





```
distance: 399.00 cm | Level: 0.00% | SAFE
distance: 399.00 cm | Level: 0.00% | SAFE
distance: 100.00 cm | Level: 0.00% | SAFE
distance: 399.00 cm | Level: 0.00% | SAFE
distance: 399.00 cm | Level: 0.00% | SAFE
distance: 398.00 cm | Level: 0.00% | SAFE
distance: 399.00 cm | Level: 0.00% | SAFE
Sending SMS...
```

SMS Sent.

```
distance: 7.00 cm | Level: 93.00% | DANGER
distance: 10.00 cm | Level: 90.00% | DANGER
distance: 100.00 cm | Level: 0.00% | SAFE
```

```
Distance: 399.00 cm | Level: 0.00% | SAFE
Distance: 100.00 cm | Level: 0.00% | SAFE
Distance: 399.00 cm | Level: 0.00% | SAFE
Distance: 399.00 cm | Level: 0.00% | SAFE
Distance: 398.00 cm | Level: 0.00% | SAFE
Distance: 399.00 cm | Level: 0.00% | SAFE
```

Sending SMS...

SMS Sent.

```
Distance: 7.00 cm | Level: 93.00% | DANGER
Distance: 10.00 cm | Level: 90.00% | DANGER
Distance: 100.00 cm | Level: 0.00% | SAFE
```